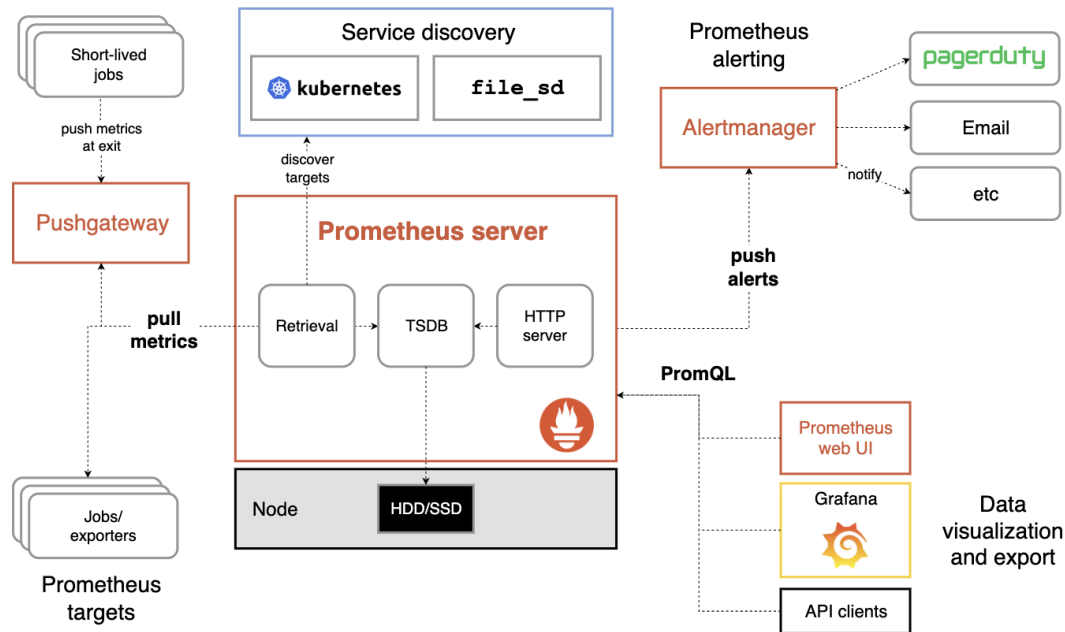


Prometheus and Grafana Assignment

Prometheus

Architecture

This diagram illustrates the architecture of Prometheus and some of its ecosystem components:



Prometheus is an open-source system monitoring and alerting toolkit. It acts as a dedicated "**Health Inspector**" for our entire infrastructure. Its main job is to collect **time series data**—which are just **numerical measurements (metrics)** recorded with a specific **timestamp**.

Grafana

Grafana is an open-source platform for monitoring, visualisation, and alerting on time-series data. Grafana offers an intuitive interface for analysing data from multiple sources (Prometheus, InfluxDB, MySQL, etc.). However, it doesn't store metrics; instead, it queries external databases and displays data through interactive dashboards.

Node Exporter collects metrics → Prometheus pulls and stores data → Grafana queries Prometheus using PromQL.

1. Setup and configure Prometheus to scrap stats (system, service) from mariadb nodes

a. Attach three volumes (5GB each) to VM(in mh6) and mount as logical volume using LVM

Step 1.1: ***multipass launch --name mon-node --disk 20G***

Step 1.2: ***multipass shell mon-node***

Step 1.3: ***lsblk***

This command lists all the storage "blocks" or hard drives the computer can see. It's like opening a map to see how many disks are plugged in and how big they are.

Step 1.4: ***sudo mkdir -p /mnt/disks***

This creates a new, specific folder (a "closet") called `/mnt/disks`. We will use this folder to store the big files that will soon act like fake hard drives.

Step 1.5: ***for i in {1..3}; do
sudo fallocate -l 5G /mnt/disks/vol\$i.img
done***

This is a loop that quickly carves out three giant 5GB files named vol1, vol2, and vol3. It's like taking a 20GB block of marble and cutting out three equal 5GB bricks to use later.

Step 1.6: ***sudo losetup /dev/loop10 /mnt/disks/vol1.img***

Step 1.7: ***sudo losetup /dev/loop11 /mnt/disks/vol2.img***

Step 1.8: ***sudo losetup /dev/loop12 /mnt/disks/vol3.img***

This tells Linux to treat these 5GB files as if they were real, physical hard drives. The computer now thinks you just plugged in three separate 5GB USB sticks named loop10, loop11, and loop12

Step 1.9: ***sudo pvcreate /dev/loop10 /dev/loop11 /dev/loop12***

This prepares your three "fake" drives to be managed by the LVM (Logical Volume Manager) system. It's like putting a special sticker on each 5GB brick saying, "This is now part of the master building set."

Step 1.10: **`sudo vgcreate vg_prometheus /dev/loop10 /dev/loop11 /dev/loop12`**

This command takes those three 5GB bricks and throws them into one big "bucket" called a Volume Group. Now, instead of three small 5GB piles, you have one giant 15GB pool of storage to work with.

Step 1.11: **`sudo lvcreate -l 100%FREE -n lv_data vg_prometheus`**

This takes all the space inside your 15GB bucket and creates one "Virtual Disk" called `lv_data`. To the computer, this looks like one single, massive 15GB hard drive instead of three small ones.

Step 1.12: **`sudo mkfs.ext4 /dev/vg_prometheus/lv_data`**

This "formats" your new 15GB virtual drive so it's ready to hold files. It's like drawing lines on a blank notebook so Prometheus knows exactly where to write its data.

`#pv,vg,lv`

`#pv display, vg display, lv display`

Physical Volume (PV): These are the **actual hard drives** or partitions that serve as the "raw materials" for your storage.

Volume Group (VG): This is a **storage pool** created by combining your disks together into one giant "virtual drive".

Logical Volume (LV): This is a **usable partition** carved out of the storage pool that you can format and store files on .

LVM takes your **physical disks (PVs)**, throws them into one **big pool (VG)**, and lets you cut out **custom-sized virtual drives (LVs)** that are easy to resize later.

Without LVM: You would have to turn off the server, copy all data to a bigger disk, and restart.

```
[ubuntu@mon-node:~$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda          8:0    0   20G  0 disk
├─sda1       8:1    0   19G  0 part /
├─sda15      8:15   0    99M  0 part /boot/efi
└─sda16     259:0   0   923M  0 part /boot
vda         253:0   0    54K  1 disk
[ubuntu@mon-node:~$ sudo mkdir -p /mnt/disks
ubuntu@mon-node:~$ for i in {1..3}; do
    sudo falloccate -l 5G /mnt/disks/vol$i.img
done
[ubuntu@mon-node:~$ sudo losetup /dev/loop10 /mnt/disks/vol1.img
[ubuntu@mon-node:~$ sudo losetup /dev/loop11 /mnt/disks/vol2.img
[ubuntu@mon-node:~$ sudo losetup /dev/loop12 /mnt/disks/vol3.img
[ubuntu@mon-node:~$ sudo pvcreate /dev/loop10 /dev/loop11 /dev/loop12
Physical volume "/dev/loop10" successfully created.
Physical volume "/dev/loop11" successfully created.
Physical volume "/dev/loop12" successfully created.
[ubuntu@mon-node:~$ sudo vgcreate vg_prometheus /dev/loop10 /dev/loop11 /dev/loop12
Volume group "vg_prometheus" successfully created
[ubuntu@mon-node:~$ sudo lvcreate -l 100%FREE -n lv_data vg_prometheus
Logical volume "lv_data" created.
[ubuntu@mon-node:~$ sudo mkfs.ext4 /dev/vg_prometheus/lv_data
mke2fs 1.47.0 (5-Feb-2023)
Discarding device blocks: done
Creating filesystem with 3929088 4k blocks and 983040 inodes
Filesystem UUID: f4f1557d-a687-4a5f-86b8-cd7b934345a3
Superblock backups stored on blocks:
    32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632, 2654208

Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Writing superblocks and filesystem accounting information: done
```

b. Configure prometheus to store data in above volume

Step 1.1: ***sudo mkdir -p /var/lib/prometheus***

It is the standard place where Prometheus expects to store all its files.

Step 1.2: ***sudo mount /dev/vg_prometheus/lv_data /var/lib/prometheus***

This "plugs" our 15GB LVM virtual disk into the folder we just created. Now, anything Prometheus saves in that folder is actually being written onto your 3-disk LVM setup.

Step 1.3: ***sudo apt update && sudo apt install -y prometheus***

This command downloads and installs the Prometheus software itself.

Step 1.4: ***sudo chown -R prometheus:prometheus /var/lib/prometheus***

This changes the "Ownership" of the folder so the Prometheus software has permission to use it. It's like handing the keys of the 15GB warehouse over to the Prometheus user so it can lock and unlock the door.

Step 1.5: ***sudo sed -i 's|ARGS="|ARGS="--storage.tsdb.path=/var/lib/prometheus |' /etc/default/prometheus***

This is done to tell Prometheus to put all the collected data in the 15GB lvm and not the default storage

Step 1.6: ***sudo systemctl daemon-reload***

Step 1.7: ***sudo systemctl restart prometheus***

Step 1.8: ***sudo systemctl status prometheus***

```
[ubuntu@mon-node:~]$ sudo systemctl status prometheus
● prometheus.service - Monitoring system and time series database
   Loaded: loaded (/usr/lib/systemd/system/prometheus.service; enabled; preset: enabled)
   Active: active (running) since Thu 2025-12-18 10:51:48 IST; 7s ago
     Docs: https://prometheus.io/docs/introduction/overview/
    Main PID: 3800 (prometheus)
      Tasks: 6 (limit: 1057)
     Memory: 13.4M (peak: 13.6M)
        CPU: 45ms
    CGroup: /system.slice/prometheus.service
            └─3800 /usr/bin/prometheus --storage.tsdb.path=/var/lib/prometheus

Dec 18 10:51:48 mon-node prometheus[3800]: ts=2025-12-18T05:21:48.312Z caller=head.go:755 level=info component=tsdb msg="WAL segment loaded" segment=0 maxSegment=0
Dec 18 10:51:48 mon-node prometheus[3800]: ts=2025-12-18T05:21:48.313Z caller=head.go:792 level=info component=tsdb msg="WAL replay completed" checkpoint_replay_duration=14.371µs wal_rep
Dec 18 10:51:48 mon-node prometheus[3800]: ts=2025-12-18T05:21:48.316Z caller=main.go:1025 level=info fs_type=EXT4_SUPER_MAGIC
Dec 18 10:51:48 mon-node prometheus[3800]: ts=2025-12-18T05:21:48.316Z caller=main.go:1028 level=info msg="TSDB started"
Dec 18 10:51:48 mon-node prometheus[3800]: ts=2025-12-18T05:21:48.316Z caller=main.go:1209 level=info msg="Loading configuration file" filename=/etc/prometheus/prometheus.yml
Dec 18 10:51:48 mon-node prometheus[3800]: ts=2025-12-18T05:21:48.317Z caller=main.go:1246 level=info msg="Completed loading of configuration file" filename=/etc/prometheus/prometheus.yml
Dec 18 10:51:48 mon-node prometheus[3800]: ts=2025-12-18T05:21:48.317Z caller=main.go:989 level=info msg="Server is ready to receive web requests."
Dec 18 10:51:48 mon-node prometheus[3800]: ts=2025-12-18T05:21:48.317Z caller=manager.go:313 level=info component=web msg="Listening on" address=[::]:9090
Dec 18 10:51:48 mon-node prometheus[3800]: ts=2025-12-18T05:21:48.317Z caller=manager.go:316 level=info component=web msg="TLS is disabled." http2=false address=[::]:9090
Dec 18 10:51:48 mon-node prometheus[3800]: ts=2025-12-18T05:21:48.317Z caller=manager.go:999 level=info component="rule manager" msg="Starting rule manager..."

[ubuntu@mon-node:~]$ df -h /var/lib/prometheus
Filesystem      Size  Used Avail Use% Mounted on
/dev/mapper/vg_prometheus-lv_data 150G 180K 140G   1% /var/lib/prometheus
```

Step 1.9: ***sudo vim /etc/prometheus/prometheus.yml***

This opens the "Address Book" where Prometheus keeps a list of who it should monitor. You are adding the IP address of your MariaDB node so Prometheus knows exactly where to go to get the data.

None

```
- job_name: 'mariadb_system'
  static_configs:
```

```
- targets: ['192.168.64.52:9100']  
- job_name: 'mariadb_service'  
static_configs:  
  - targets: ['192.168.64.52:9104']
```

Step 1.10: ***sudo systemctl restart prometheus***

Step 1.11: ***sudo systemctl status prometheus***

2. Setup and configure Grafana to represent graph of system and service level metrics

Step 2.1: ***sudo apt-get install -y apt-transport-https software-properties-common wget***

This command installs a few small "helper tools" that your computer needs to talk to the internet securely. It's like making sure your computer has the right browser and security certificates before it tries to download a new program.

Step 2.2: ***sudo mkdir -p /etc/apt/keyrings***

This creates a special, secure "key cabinet" folder on your system. We use this folder to store digital "keys" that prove the software we are about to download is official and safe.

Step 2.3: ***wget -q -O - https://apt.grafana.com/gpg.key | gpg --dearmor | sudo tee /etc/apt/keyrings/grafana.gpg > /dev/null***

This downloads the official "Security Key" from the Grafana company and locks it in your key cabinet. It's like getting a signed letter from the manufacturer so your computer knows it's not downloading a virus.

Step 2.4: ***echo "deb [arch=arm64 signed-by=/etc/apt/keyrings/grafana.gpg] https://apt.grafana.com stable main" | sudo tee /etc/apt/sources.list.d/grafana.list***

This tells your computer exactly where the Grafana software "store" is located on the internet. By adding this to your sources list, you're giving your computer the "web address" of the official Grafana download center.

Step 2.5: ***sudo apt-get update***

Step 2.6: ***sudo apt-get install -y grafana***

This is the main event: it actually downloads and installs the Grafana "Artist" software.

Step 2.7: ***sudo systemctl daemon-reload***

Step 2.8: ***sudo systemctl enable --now grafana-server***

Step 2.9: ***sudo systemctl status grafana-server***

```
no VM guests are running outdated hypervisor (qemu) binaries on this host.
(ubuntu@non-node:~)$ sudo systemctl daemon-reload
(ubuntu@non-node:~)$ sudo systemctl enable --now grafana-server
Synchronizing state of grafana-server.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable grafana-server
Created symlink /etc/systemd/system/multi-user.target.wants/grafana-server.service → /usr/lib/systemd/system/grafana-server.service.
(ubuntu@non-node:~)$ sudo systemctl status grafana-server
● grafana-server.service - Grafana instance
   Loaded: loaded (/usr/lib/systemd/system/grafana-server.service; enabled; preset: enabled)
   Active: active (running) since Thu 2025-12-18 11:06:48 IST; 7s ago
     Docs: http://docs.grafana.org
   Main PID: 5256 (grafana)
    Tasks: 7 (limit: 1867)
  Memory: 309.1M (peak: 309.4M)
     CPU: 2.541s
   CGroup: /system.slice/grafana-server.service
           └─5256 /usr/share/grafana/bin/grafana server --config=/etc/grafana/grafana.ini --pidfile=/run/grafana/grafana-server.pid --packaging=deb cfg:default.paths.logs=/var/log/grafana cfg:default.paths.data=/var/lib/grafana cfg:default.paths.plugins=/var/lib/grafana/plugins cty
Dec 18 11:06:55 non-node grafana[5256]: logger=plugin.installer t=2025-12-18T11:06:55.198383331+05:30 level=info msg="Installing plugin" pluginId=grafana-explorettraces-app version=
Dec 18 11:06:55 non-node grafana[5256]: logger=installer.fs t=2025-12-18T11:06:55.256282548+05:30 level=info msg="Downloaded and extracted grafana-explorettraces-app v1.2.2 zip successfully to /var/lib/grafana/plugins/grafana-explorettraces-app"
Dec 18 11:06:55 non-node grafana[5256]: logger=plugins.registration t=2025-12-18T11:06:55.267981344+05:30 level=info msg="Plugin registered" pluginId=grafana-explorettraces-app
Dec 18 11:06:55 non-node grafana[5256]: logger=plugin.backgroundinstaller t=2025-12-18T11:06:55.268836378+05:30 level=info msg="Plugin successfully installed" pluginId=grafana-explorettraces-app version= duration=888.342800ms
Dec 18 11:06:55 non-node grafana[5256]: logger=plugin.backgroundinstaller t=2025-12-18T11:06:55.268891867+05:30 level=info msg="Installing plugin" pluginId=grafana-metricsdrilldown-app version=
Dec 18 11:06:55 non-node grafana[5256]: logger=plugin.installer t=2025-12-18T11:06:55.285826962+05:30 level=info msg="Installing plugin" pluginId=grafana-metricsdrilldown-app version=
Dec 18 11:06:56 non-node grafana[5256]: logger=installer.fs t=2025-12-18T11:06:56.061989427+05:30 level=info msg="Downloaded and extracted grafana-metricsdrilldown-app v1.0.26 zip successfully to /var/lib/grafana/plugins/grafana-metricsdrilldown-app"
Dec 18 11:06:56 non-node grafana[5256]: logger=plugins.registration t=2025-12-18T11:06:56.07722318+05:30 level=info msg="Plugin registered" pluginId=grafana-metricsdrilldown-app
Dec 18 11:06:56 non-node grafana[5256]: logger=plugin.backgroundinstaller t=2025-12-18T11:06:56.077859434+05:30 level=info msg="Plugin successfully installed" pluginId=grafana-metricsdrilldown-app version= duration=889.76174ms
Dec 18 11:06:56 non-node grafana[5256]: logger=plugin.backgroundinstaller t=2025-12-18T11:06:56.078813464+05:30 level=info msg="Installing plugin" pluginId=grafana-lokiexplore-app version=
lines 1-21/21 (END)
```

In maria db node run these commands

Step 2.10: ***sudo apt update***

Step 2.11: ***sudo apt install -y prometheus-node-exporter prometheus-mysqld-exporter***

This command installs two "translators" (Exporters) on the database server. One translator speaks for the computer's health (CPU/RAM), and the other speaks for the MariaDB database's health.

Step 2.12: ***sudo mysql -u root -p***

Step 2.13: ***CREATE USER 'exporter'@'localhost' IDENTIFIED BY 'admin';***

This opens the database and creates a new "Security Guard" user named **exporter**. We give this guard a password ('admin') so only authorized monitoring tools can log in and see the stats.

Step 2.14: ***GRANT PROCESS, REPLICATION CLIENT, SELECT ON *.* TO 'exporter'@'localhost';***

This gives the "Security Guard" a very specific set of keys that only open the "Health Report" cabinets. It allows the exporter to see how fast the database is running, but it cannot see your private data or customer tables.

Step 2.15: ***FLUSH PRIVILEGES;***

This tells the database to "save and refresh" the new security rules you just created. Once refreshed, you leave the database command center and go back to the normal terminal.

Step 2.16: ***EXIT;***

Step 2.17: ***sudo vim /var/lib/prometheus/.my.cnf***

This creates a hidden "ID Card" (configuration file) where we write down the guard's username and password. This way, the exporter can look at this card to log in automatically without you having to type the password every time.

```
None
[client]
user=exporter
password=admin
host=localhost
```

Step 2.18: ***sudo chown prometheus:prometheus /var/lib/prometheus/.my.cnf***

Step 2.19: ***sudo chmod 600 /var/lib/prometheus/.my.cnf***

These commands lock the "ID Card" so that only the Exporter service can read it. It's like putting the ID card in a safe that only the authorized monitoring person can open—this keeps your password secure from others.

Step 2.20: ***sudo vim /etc/default/prometheus-mysqld-exporter***

This opens the "Instruction Manual" for the database translator. You are adding a line that says: "Hey, when you want to log in, go look at the secret ID card we just locked in the safe."

```
None
ARGS="--config.my-cnf=/var/lib/prometheus/.my.cnf"
```

Step 2.21: ***sudo systemctl restart prometheus-mysqld-exporter***

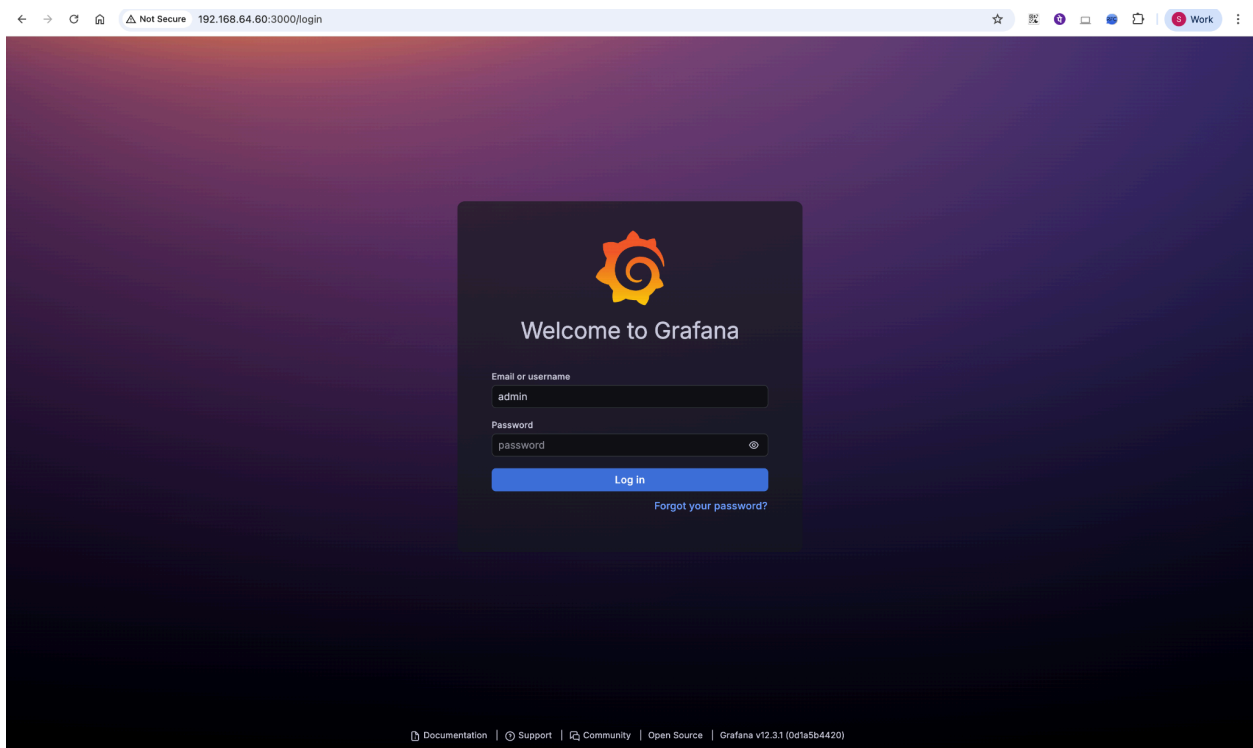
Step 2.22: ***sudo systemctl enable --now prometheus-node-exporter***

Step 2.23: **`curl http://localhost:9100/metrics`**

Check Browser

Now that the service is running, we will check in the browser

1. **URL:** `http://<VM_IP>:3000`
2. **Credentials:** * **User:** `admin`
 - **Password:** `admin` (It will immediately ask you to set a new one).



Add Prometheus as a Data Source

Grafana needs to know where Prometheus is storing the metrics (on our LVM volume).

1. On the left sidebar, click the **Connections** (three-line icon or gear) and select **Data sources**.
2. Click **Add data source**.
3. Select **Prometheus**.
4. In the **Connection** section, find the **URL** field and enter: `http://localhost:9090`

5. Scroll to the bottom and click **Save & test**.
 - we should see a green checkmark saying: *"Data source is working"*.

1. Import the System Metrics Dashboard

This will show you CPU usage, RAM consumption, and Disk I/O (stored on your LVM volume).

1. In the Grafana sidebar, click **Dashboards** (the square grid icon).
2. Click the **New** button and select **Import**.
3. In the box "Import via grafana.com", enter ID: **1860** (Node Exporter) and click **Load**.
4. On the next screen:
 - Give it a name (e.g., "System Stats - LVM VM").
 - Select your **Prometheus** data source in the dropdown at the bottom.
5. Click **Import**.

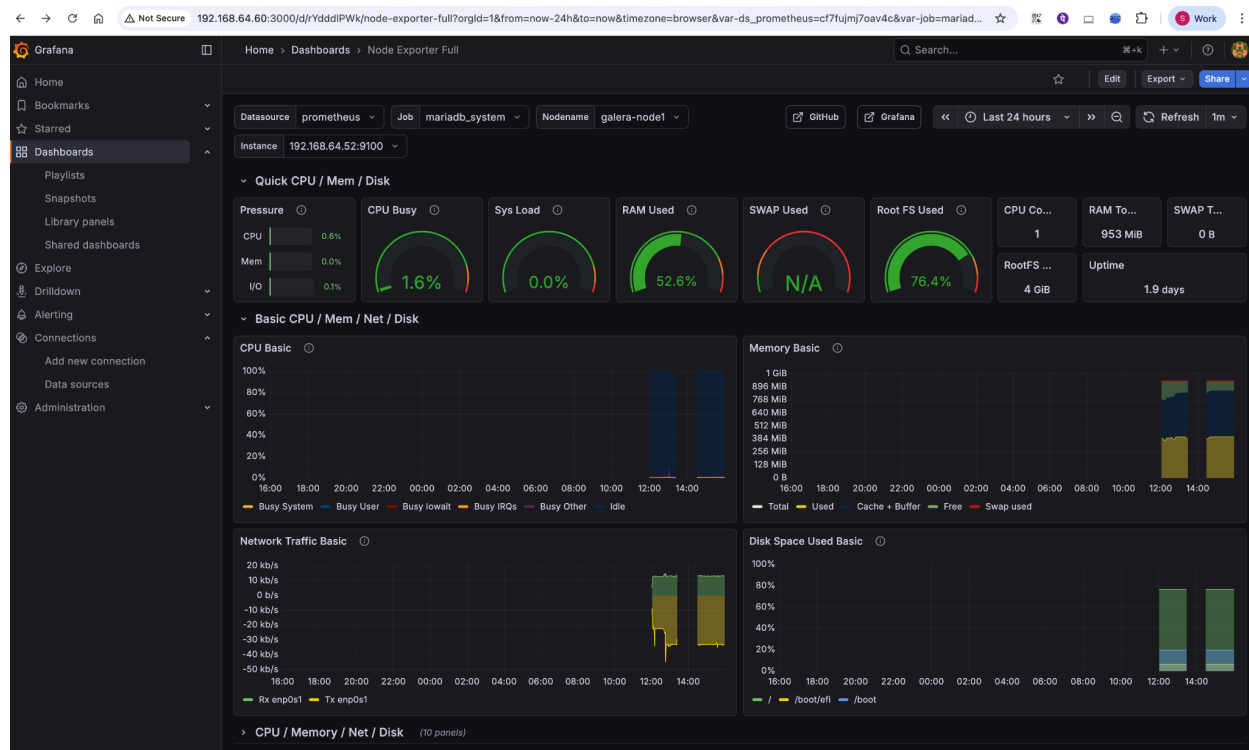
2. Import the MariaDB Metrics Dashboard

This will show our database-specific stats like queries per second, active connections, and buffer pool usage.

1. Go back to **Dashboards > New > Import**.
2. In the "Import via grafana.com" box, enter ID: **14057** (14057) and click **Load**.
3. On the next screen:
 - Ensure the data source is set to **Prometheus**.
4. Click **Import**.

The completes the monitoring "Stack":

1. **MariaDB**: Generates data.
2. **Exporters**: (Usually **node_exporter** and **mysqld_exporter**) translate that data into a format Prometheus understands.
3. **Prometheus**: Scrapes that data and saves it onto your **LVM volume**.
4. **Grafana**: Reaches into Prometheus and turns those numbers into the pretty charts you just imported.



<http://192.168.64.60:9090/classic/targets>

Prometheus Alerts Graph Status Help					
Targets					
All Unhealthy Collapse All					
mariadb_service (1/1 up) show less					
Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://192.168.64.52:9104/metrics	UP	instance="192.168.64.52:9104" job="mariadb_service"	11.85s ago	52.25ms	
mariadb_system (1/1 up) show less					
Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://192.168.64.52:9100/metrics	UP	instance="192.168.64.52:9100" job="mariadb_system"	11.636s ago	75.79ms	
node (1/1 up) show less					
Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://localhost:9100/metrics	UP	instance="localhost:9100" job="node"	13.648s ago	105.6ms	
prometheus (1/1 up) show less					
Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://localhost:9090/metrics	UP	instance="localhost:9090" job="prometheus"	4.386s ago	14.26ms	